

DNS Tunneling & Detection

Research Report: Mechanics, Attack Patterns, and Deep Packet Inspection

The Blind Spot: Port 53

Traditional firewalls operate primarily at **Layer 3 and Layer 4**, checking the source IP, destination IP, and ports, without looking inside the payload.

Because translating domains is essential for nearly all network activity, firewalls are universally configured to inherently trust and allow traffic flowing through **Port 53**. Adversaries exploit this blind spot. By wrapping malware and stolen data inside normal-looking DNS queries, they establish a covert channel that bypasses sophisticated perimeter defenses completely undetected.



How the Exploit Works

2. Trusted Query

The machine asks a trusted local DNS resolver (e.g., 8.8.8.8) to resolve the fake domain.

4. Data Extracted

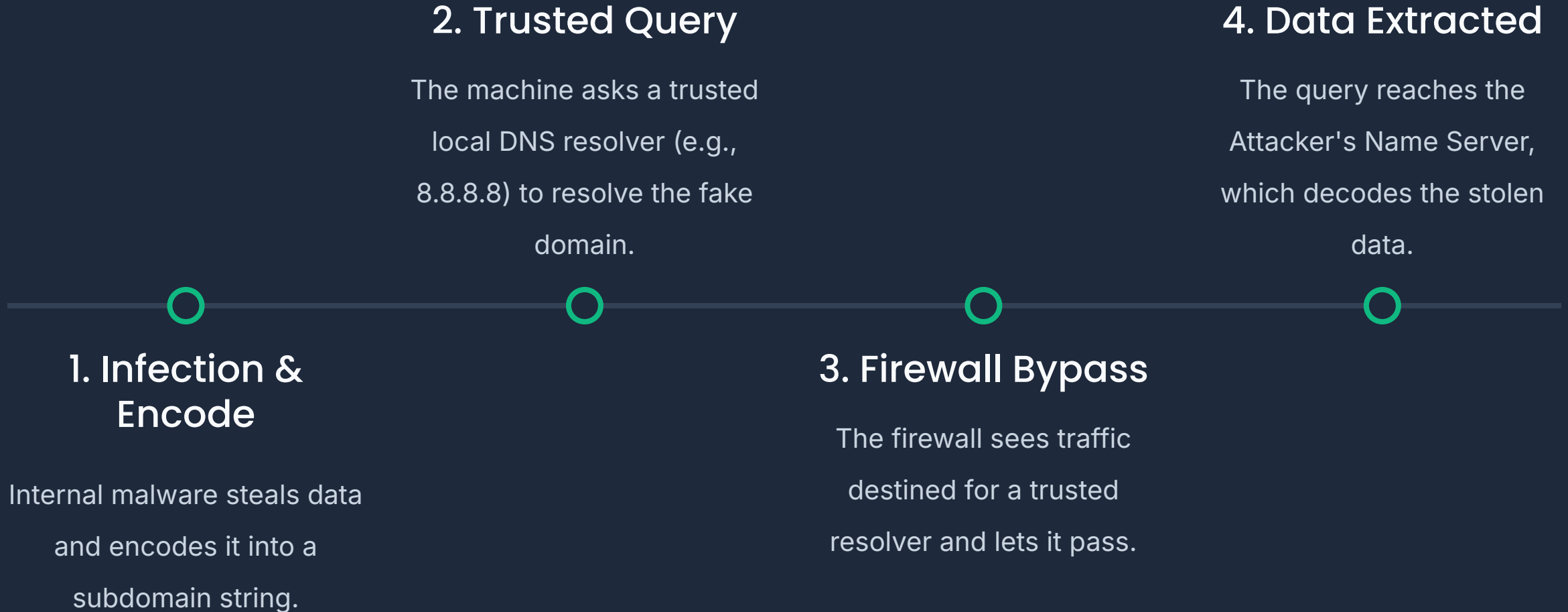
The query reaches the Attacker's Name Server, which decodes the stolen data.

1. Infection & Encode

Internal malware steals data and encodes it into a subdomain string.

3. Firewall Bypass

The firewall sees traffic destined for a trusted resolver and lets it pass.



Common Attack Patterns



Data Exfiltration

Attackers chunk and encode stolen data (e.g., Base64) into subdomains to adhere to strict DNS protocol length limits, slipping documents past DLP systems.



Command & Control

Instead of standard 'A' records, malware requests 'TXT' or 'NULL' records which can hold up to 255 characters, pulling encrypted commands back to the victim.



Low & Slow Evasion

To bypass volume filters and mathematical checks, adversaries send extremely short strings over long, stretched-out periods to mimic regular traffic.

Weaponized Tools

Modern attackers utilize robust, open-source tunneling software to create highly resilient and fully encrypted persistent connections.

DNScat2

Specifically designed to establish an encrypted C&C channel. It heavily utilizes automated "beaconing" techniques to maintain a continuous, hidden connection with the adversary.

Iodine

A powerful tool built to tunnel entire IPv4 data streams through a DNS server. This effectively grants the attacker a full VPN-like connection directly into the compromised network.

PROCESS MANAGEMENT

ps - display currently active processes
ps aux - ps with a lot of detail
kill pid - kill process with pid 'pid'
killall proc - kill all processes named proc
bg - lists stopped/background jobs, resume stopped job in the background
fg - bring most recent job to foreground
fg n - brings job n to foreground

FILE PERMISSIONS

chmod octal file - change permission of file

```
file 4 - read (r)
      2 - write (w)
      1 - execute (x)

order: owner/group/world

eg:
chmod 777 - rwx for everyone
chmod 755 - rw for owner, rx for group/world
```

COMPRESSION

```
tar cf file.tar files - tar files into file.tar
tar xf file.tar - untar into current directory
tar tf file.tar - show contents of archive
```

tar flags:

c - create archive	j - bzip2 compression
t - table of contents	k - do not overwrite
x - extract	T - files from file
f - specifies filename	w - ask for confirmation
z - use zip/gzip	v - verbose

gzip file - compress file and rename to file.gz
gzip -d file.gz - decompress file.gz

SHORTCUTS

```
ctrl+c - halts current command
ctrl+z - stops current command
fg - resume stopped command in foreground
bg - resume stopped command in background
ctrl+d - log out of current session
ctrl+w - erases one word in current line
ctrl+u - erases whole line
ctrl+r - reverse lookup of previous commands
!! - repeat last command
exit - log out of current session
```

VIM

quitting

!x - exit, saving changes
!wq - exit, saving changes
!q - exit, if no changes
!q! - exit, ignore changes

inserting text

i - insert before cursor
I - insert before line
a - append after cursor
A - append after line
o - open new line after cur line
O - open new line before cur line
r - replace one character
R - replace many characters

VIM

motion

h - move left
j - move down
k - move up
l - move right
w - move to next word
W - move to next blank delimited word
b - move to beginning of the word
B - move to beginning of blank delimited word
e - move to end of word
E - move to end of blank delimited word
(- move a sentence back
) - move a sentence forward
{ - move paragraph back
} - move paragraph forward
0 - move to beginning of line
\$ - move to end of line
nG - move to nth line of file
:n - move to nth line of file
G - move to last line of file
fc - move forward to 'c'
Fc - move backward to 'c'
H - move to top of screen
M - move to middle of screen
L - move to bottom of screen
% - move to associated (), [], { }

deleting text

x - delete character to the right
X - delete character to the left
D - delete to the end of line
dd - delete current line
:d - delete current line

searching

/string - search forward for string
?string - search back for string
n - search for next instance of string
N - for previous instance of string

replace

:s/pattern/string/flags - replace pattern with string, according to flags
g - flag, replace all occurrences
c - flag, confirm replaces
A - repeat last :s command

files

:w file - write to file
:r file - read file in after line
:n - go to next file
:p - go to previous file
:e file - edit file
!cmd - replace line with output of cmd

other

u - undo last change
U - undo all changes to line

Defense in Depth Architecture

 **Lexical Analysis** Mathematical anomaly detection used to evaluate the randomness of subdomain characters. Encoded payloads lose human linguistic patterns, generating highly unnatural scores.

 **Protocol Feature Analysis** Leveraging Deep Packet Inspection (DPI) to monitor the structural intent of queries. Flags are raised when excessive TXT, CNAME, or NULL records are requested internally.

 **Traffic & Volume Tracking** Behavioral tracking that analyzes query frequency and inter-arrival times. By measuring the standard deviation between requests, robotic beaconing habits are easily exposed.

Lexical Analysis: **Shannon Entropy**

To identify encrypted data hiding within subdomains, we apply **Shannon Entropy** to measure the degree of randomness (chaos) in the string.

In natural language (legitimate domains), characters frequently repeat, resulting in a low entropy score. Conversely, Base64 encoded malware payloads contain distinct, non-repeating characters, causing the entropy score to spike significantly.

$$H(X) = - \sum_{i=1}^k P(x_i) \log_2 P(x_i)$$

Dual-Engine Logic Evaluation

To eliminate false positives from naturally long Vietnamese domains, the Python Analyzer enforces dual requirements:
Length $\geq$ 15 AND Entropy $\geq$ 3.5.

Subdomain Extracted	Length	Entropy Score	System Evaluation
translate	9	2.948	SAFE
congtacvien	11	3.096	SAFE
bWF0X2toYXU=	12	3.585	MONITORING REQUIRED
ZGF5LWxhLW1hdC1raGF1	20	3.840	MALWARE ALERT

Future Work & Next Steps

While static lexical analysis is highly effective against brute-force data exfiltration, combating 'low and slow' evasion tactics requires adding temporal context.

DPI Integration

The next phase involves integrating the Python analyzer with Deep Packet Inspection engines like Suricata to continuously parse stateful eve.json logs.

Behavioral Tracking

By tracking query frequency and calculating the standard deviation of inter-arrival times per IP address, we will fuse volume-based behavioral tracking with

